



**The Faculty of Information and Communication Technology
Mahidol University**

**Project 3: Logical Database Design
E-Commerce: IKEA**

Group ID - G01

Mr.	Punnawit	Kanogpornwanich	6688042
Mr.	Bhumipat	Pengpaiboon	6688077
Mr.	Napas	Siripaskrittakul	6688108

ITCS442 Relational Database Design
Asst.Prof. Dr. Charnyote Pluempitiwiriyawej
February 25, 2026

Table of Contents

Business Domain and Problem statement	3
Real Business Company	3
Problem statement	3
Declaration of AI tool usage	3
Overall Project Phase 3	4
What we have done in Project Phase 3	4
Latest UML Diagram	5
Mapping to Relational Schema	6
Relational Schema Description	7
IKEA Normalization	8
Relational Schema After Normalization	13
UML Diagram After Normalization	14
What we have changed from Project Phase 2	15
Reference	16

Business Domain and Problem statement

Real Business Company: IKEA (Customer and Order Management System)

IKEA Thailand provides an omnichannel retail service including online ordering, click & collect, delivery service, installation service, assembly service, etc. Customers can use the IKEA website to track their orders and schedule to access the services, while IKEA staff are responsible for the process such as preparing orders, scheduling deliveries, and arranging booking services. However, operational data for each section is separated across different systems and workflows, such as tracking delivery, furniture assembly services, and other services. Each service has different conditions depending on the specifications of the product, resulting in an increased amount of data and the need for continuous status updates.

Problem statement

This project proposes the development of a centralized database application that combines order fulfillment, service booking, and return management into a single system. This system aims to reduce operational errors of scattered data and improve data connectivity across different services.

Declaration of AI tool usage : AI Level 2

This project used AI tools (ChatGPT, Gemini) as a supporting assistant for ideas, improving English writing (paraphrasing, grammar correction, and clarity), and organizing report structure.

All database design decisions (e.g. key characteristics, mission statement, mission object, scope and boundary diagram, business function, and requirement specification) were reviewed and verified by everyone in group 1. The final part of analysis, implementation, and report in phase 1 were completed and finalized by team members. We take full responsibility for the accuracy, originality, and integrity of the submitted work.

Overall project Phase 3

Project Phase 3 focuses on transforming the conceptual database design (UML diagram) that was developed in Phase 2 into a structured logical database model for the IKEA Customer and Order Management System. Phase 3 formalizes these components into a relational schema that accurately represents IKEA Thailand's online, including order processing, inventory management, membership care, shipment tracking, and return and refund handling.

What we have done in Project Phase 3

In this phase, all major entities such as Customer, Order, Orderline, Product, Payment, Transaction, Shipment, Tracking, Membership, Promotion, Stock, Branch, Return Request, and Refund Request are structured into relational schema with clearly defined primary keys (PK) and foreign keys (FK).

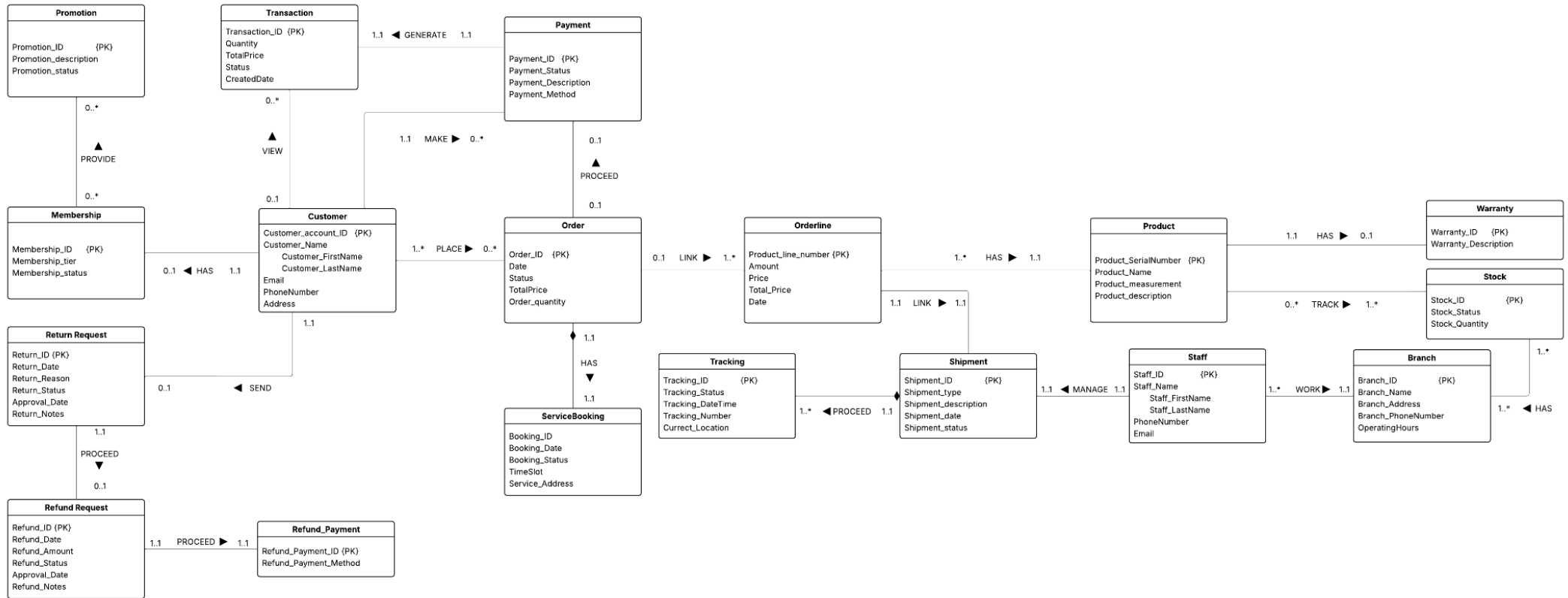
Each entity is mapped from the UML diagram into a corresponding relational table while keeping cardinality and relationship constraints. For example, One-to-many (1:N) relationships, such as Customer-Order and Order-Orderline, are implemented by using foreign key references. Weak entities, such as Tracking and ServiceBooking, are represented to maintain the correctness of integrity.

In this Project, we also applied the normalization to ensure data consistency and reduce the redundancy. Also the repeating attributes such as change Log_history to Order_history and splitting out from attribute to entity to satisfy First Normal Form (1NF). This ensures that each column stores a single piece of information and that no multiple values are stored in one attribute. Multi-valued or repeated data are properly separated into related tables to maintain consistency and clarity.

Overall, Project Phase 3 establishes a structured and consistent logical database design for the IKEA Customer and Order Management System. By transforming the conceptual UML diagram into the relational schema with clearly defined keys and relationships constraints, our system ensures data integrity and logical consistency. In this project, we apply the concept of normalization to reduce redundancy and improve data organization. Also preparing for physical database implementation in the next phase development.

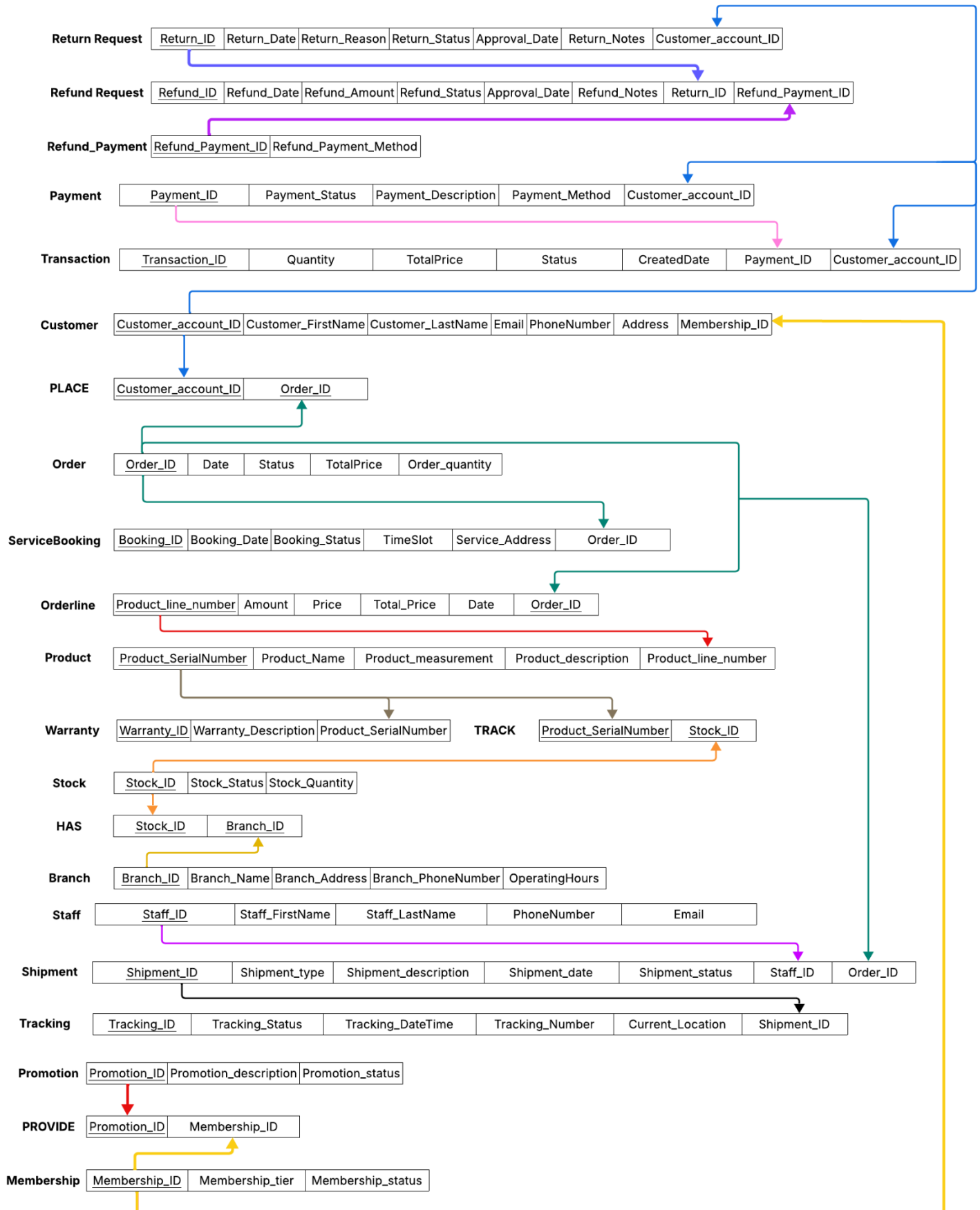
Latest UML Diagram

IKEA ; Customer and Order Management



Mapping to Relational Schema

IKEA ; Customer and Order Management



Relational Schema Description

Entity	Attribute	Link to
Customer	Customer_account_ID, Customer_FirstName, Customer_LastName, Email, PhoneNumber, Address, Membership_ID(FK)	Customer => Transaction, Customer => Payment, Customer => Return request, Customer => PLACE
Transaction	Transaction_ID, Quantity, TotalPrice, Status, CreatedDate, Payment_ID(FK), Customer_account_ID(FK)	—
Payment	Payment_ID, Payment_Status, Payment_Description, Payment_Method, Customer_account_ID(FK)	—
Order	Order_ID, Date, Status, TotalPrice, Order_quantity	Order => ServiceBooking, Order => Orderline, Order => Shipment, Order => PLACE
ServiceBooking	Booking_ID, Booking_Date, Booking_Status, TimeSlot, Service_Address, Order_ID(FK)	—
Orderline	Product_line_number, Amount, Price, Total_Price, Date, Order_ID(FK)	Orderline => Product
Product	Product_SerialNumber, Product_Name, Product_measurement, Product_description, Product_line_number(FK)	Product => Warranty, Product => TRACK
Warranty	Warranty_ID, Warranty_Description, Product_SerialNumber(FK)	—
Stock	Stock_ID, Stock_Status, Stock_Quantity	Stock => TRACK, Stock => HAS
Branch	Branch_ID, Branch_Name, Branch_Address, Branch_PhoneNumber, OperatingHours	Branch => HAS
Staff	Staff_ID, Staff_FirstName, Staff_LastName, PhoneNumber, Email	Staff => Shipment
Shipment	Shipment_ID, Shipment_type, Shipment_description, Shipment_date, Shipment_status, Staff_ID(FK), Order_ID(FK)	Shipment => Tracking
Tracking	Tracking_ID, Tracking_Status, Tracking_DateTime, Tracking_Number, Current_Location, Shipment(FK)	—
Promotion	Promotion_ID, Promotion_description, Promotion_status	Promotion => PROVIDE
Membership	Membership_ID, Membership_tier, Membership_status	Membership => Customer Membership => PROVIDE
Refund_Payment	Refund_Payment_ID, Refund_Payment_Method	Refund_Payment => Refund Request
Refund Request	Refund_ID, Refund_Date, Refund_Amount, Refund_Status, Approval_Data, Refund_Notes, Return_ID, Refund_Payment_ID(FK)	—
Return Request	Return_ID, Return_Date, Return_Reason, Return_Status, Approval_Date, Return_Notes, Customer_account_ID(FK)	Return Request => Refund Request

IKEA Normalization

Relation	Attribute	Key	Functional Dependency	Relational Schema
Customer	Customer_account_ID (PK), Customer_FirstName, Customer_LastName, Email, PhoneNumber, Address, Membership_ID (FK)	- Customer_account_ID (PK) - Membership_ID (FK)	Repeating Group (1NF): Change “Customer_account_ID” to Customer_ID to prevent miss understanding Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Customer_ID(PK) => Customer_FirstName, Customer_LastName, Email, PhoneNumber, Address	R1) Customer (Customer_ID (PK), Customer_FirstName, Customer_LastName, Email, PhoneNumber, Address)
Transaction	Transaction_ID (PK), Quantity, TotalPrice, Status, CreatedDate, Payment_ID(FK), Customer_account_ID(FK)	- Transaction_ID (PK) - Payment_ID (FK), Customer_account_ID (FK)	Repeating Group (1NF): “ Remove “TotalPrice” from Transaction because it’s derive value ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Transaction_ID(PK) => Quantity, Status, CreatedDate	R1) Transaction (Transaction_ID (PK), Quantity, Status, CreatedDate)
Payment	Payment_ID (PK), Payment_Status, Payment_Description, Payment_Method, Customer_account_ID(FK)	- Payment_ID (PK) - Customer_account_ID (FK)	Repeating Group (1NF): “Separate Payment_Method and Payment_history out of Payment” * Payment_ID(PK), Payment_Method => Payment_Status, Payment_history, Payment_Description Partial Dependency (2NF): “ Declare Attribute on Payment_history and Payment_method ” * Payment_ID(PK) => Payment_Status, Payment_history, Payment_Description * Payment_Method => Payment_type(PK), QR_code_ID(FK), Credit_Card_number(FK) Transitive Dependency (3NF): “ Add PK and FK on Payment, Payment_Method, and Payment_history ” * Payment_ID(PK) => Payment_Status, Payment_Description, , Payment_type(FK), Payment_History_ID(FK) * Payment_type(PK) => Method_Name, Description	R1) Payment (Payment_ID(PK), Payment_Status, Payment_Description, Payment_Method_ID(FK), Payment_history_ID(FK)) R2) Payment_Method (Payment_type, QR_code_ID(FK), Credit_Card_number(FK)) R2.1) Payment_QR (QR_code_ID(PK), Payment_name, Payment_description) R2.2)Payment_CreditCard (Credit_Card_number(PK), Payment_name, Payment_description,)

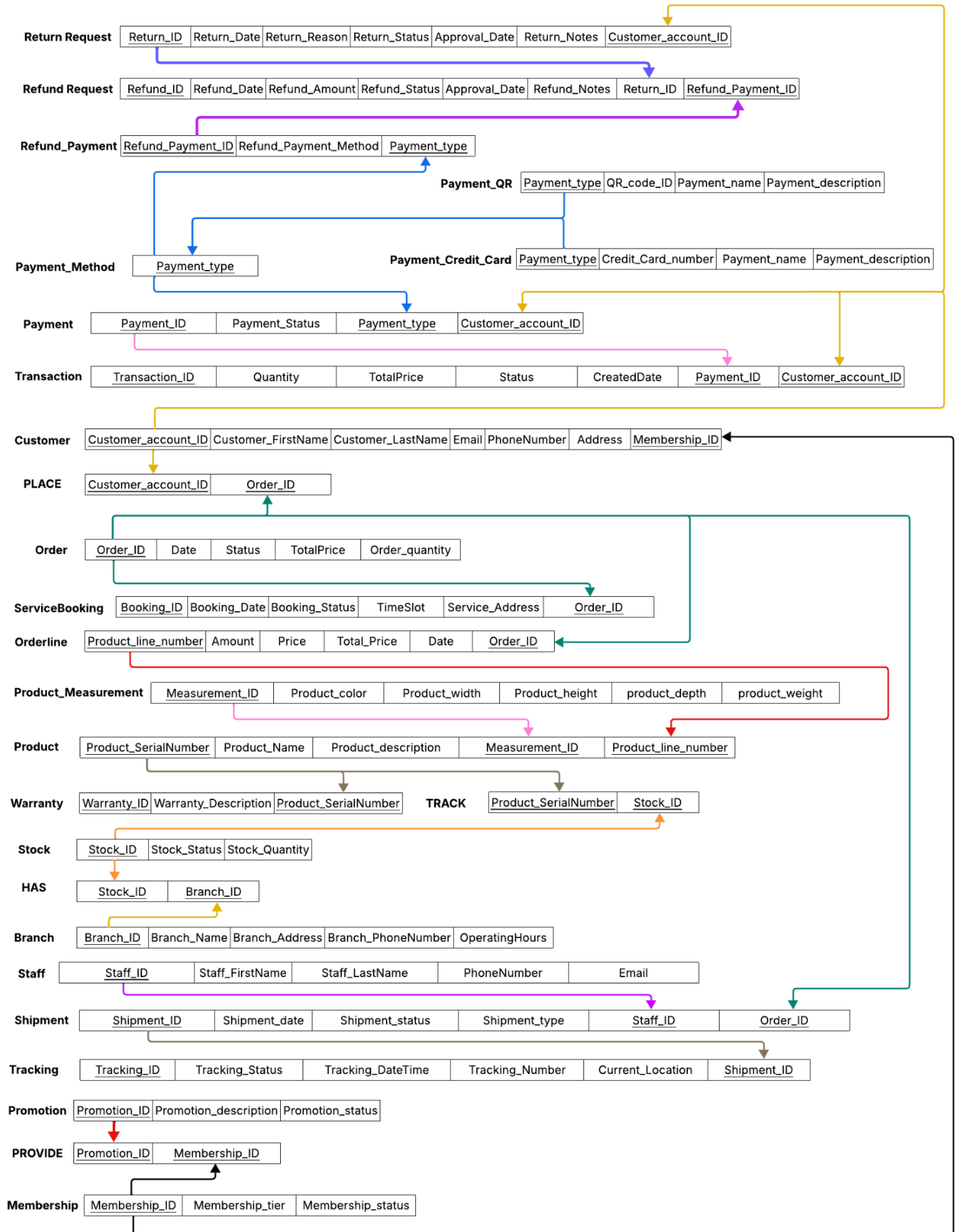
Order	Order_ID (PK), Date, Status, TotalPrice, Order_quantity	Order_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Order_ID(PK) => Date, Status, TotalPrice, Order_quantity	R1) Order (Order_ID(PK), Date, Status, TotalPrice, Order_quantity,)
ServiceBooking	Booking_ID (PK), Booking_Date, Booking_Status, TimeSlot, Service_Address, Order_ID(FK)	- Booking_ID (PK) - Order_ID(FK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Booking_ID(PK) => Booking_Date, Booking_Status, TimeSlot, Service_Address	R1) Booking (Booking_ID(PK), Booking_Date, Booking_Status, TimeSlot, Service_Address)
Orderline	Product_line_number (PK), Amount, Price, Total_Price, Date, Order_ID (FK)	- Product_line_number (PK) - Order_ID (FK)	Repeating Group (1NF): “ Remove “TotalPrice” from Orderline because it’s derive value ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Product_line_number (PK) => Amount, Price, Date	R1) Orderline (Product_line_number (PK), Amount, Price, Total_Price, Date)
Product	Product_SerialNumber (PK), Product_Name, Product_measurement, Product_description, Product_line_number(FK)	- Product_SerialNumber (PK) - Product_line_number (FK)	Repeating Group (1NF): “ Separate Product_Measurement from the Product ” * Product_SerialNumber(PK), Product_Measurement => Product_Name, Product_description Partial Dependency (2NF): “ Declare Attribute on Product_Measurement” * Product_SerialNumber(PK) => Product_Name, Product_description * Product_Measurement => Product_dimension, Product_height, Product_weight Transitive Dependency (3NF): “ Add PK and FK on Product_Measurement” * Product_SerialNumber(PK) => Product_Name, Product_description, Measurement_ID(FK)	R1) Product (Product_SerialNumber(PK), Product_Name, Product_description, Measurement_ID(FK)) R2)Product_Measurement (Measurement_ID(PK), Product_color, Product_width, Product_height, Product_weight, Product_depth)

			* Measurement_ID(PK) => Product_color, Product_width, Product_height, Product_weight, Product_depth	
Warranty	Warranty_ID (PK), Warranty_Description, Product_SerialNumber(FK)	- Warranty_ID (PK) - Product_SerialNumber (FK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Warranty_ID(PK) => Warranty_Description	R1) Warranty (Warranty_ID (PK), Warranty_Description)
Stock	Stock_ID (PK), Stock_Status, Stock_Quantity	- Stock_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Stock_ID(PK) => Stock_Status, Stock_Quantity	R1) Stock (Stock_ID (PK), Stock_Status, Stock_Quantity)
Branch	Branch_ID (PK), Branch_Name, Branch_Address, Branch_PhoneNumber, OperatingHours	- Branch_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Branch_ID(PK) => Branch_Name, Branch_Address, Branch_PhoneNumber, OperatingHours	R1) Branch (Branch_ID (PK), Branch_Name, Branch_Address, Branch_PhoneNumber, OperatingHours)
Staff	Staff_ID (PK), Staff_FirstName, Staff_LastName, PhoneNumber, Email	- Staff_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Staff_ID(PK) => Staff_FirstName, Staff_LastName, PhoneNumber, Email	R1) Staff (Staff_ID (PK), Staff_FirstName, Staff_LastName, PhoneNumber, Email)
Shipment	Shipment_ID (PK),	- Shipment_ID (PK)	Repeating Group (1NF):	R1) Shipment (Shipment_ID (PK), Shipment_type,

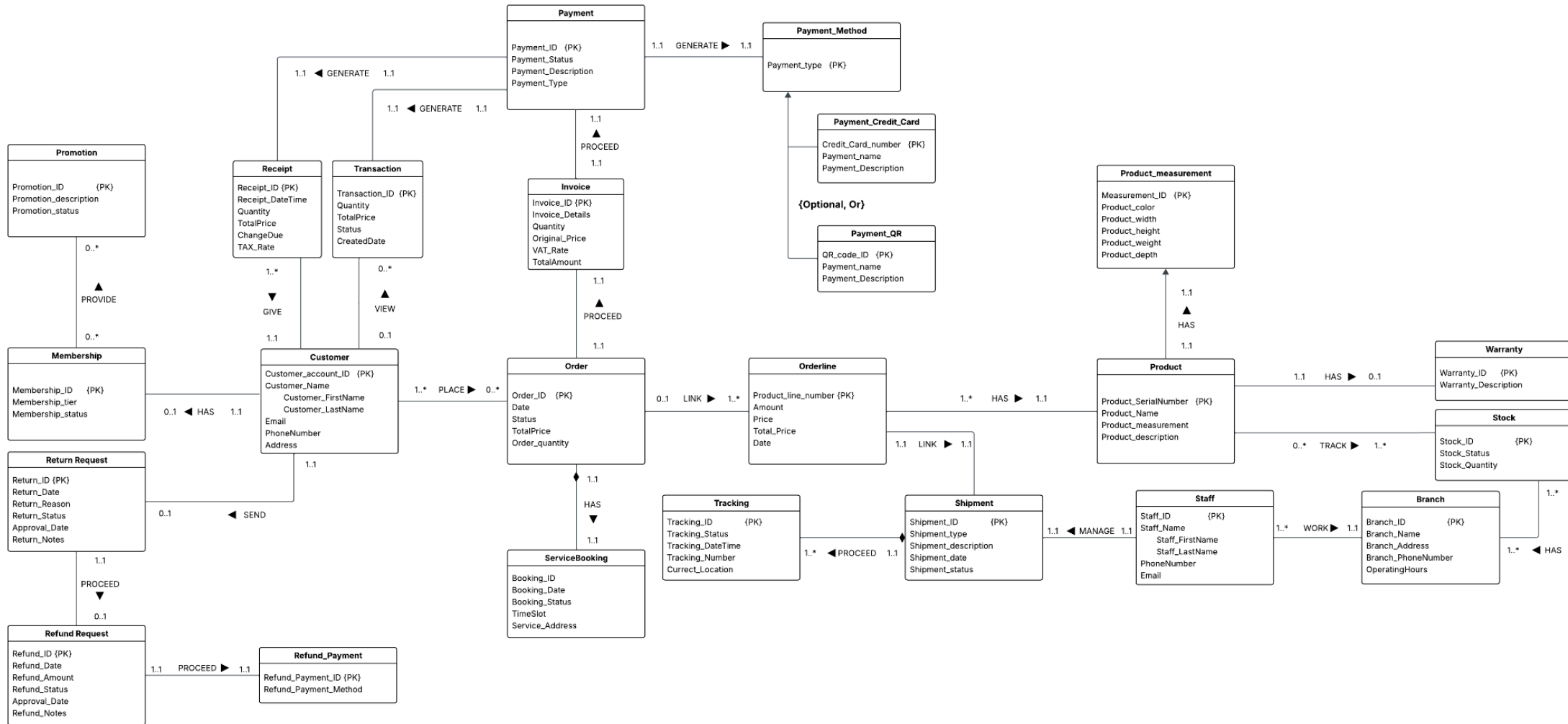
	Shipment_type, Shipment_description, Shipment_date, Shipment_status, Staff_ID(FK), Order_ID(FK)	- Staff_ID(FK) - Order_ID(FK)	“ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Shipment_ID (PK) => Shipment_type, Shipment_description, Shipment_date	Shipment_description, Shipment_date)
Tracking	Tracking_ID (PK), Tracking_Status, Tracking_DateTime, Tracking_Number, Current_Location, Shipment(FK)	- Tracking_ID (PK) - Shipment(FK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Tracking_ID(PK) => Tracking_Status, Tracking_DateTime, Tracking_Number, Current_Location	R1) Track (Tracking_ID (PK), Tracking_Status, Tracking_DateTime, Tracking_Number, Current_Location)
Promotion	Promotion_ID (PK), Promotion_description, Promotion_status	- Promotion_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Promotion_ID(PK) => Promotion_description, Promotion_status	R1) Promotion (Promotion_ID (PK), Promotion_description, Promotion_status)
Membership	Membership_ID (PK), Membership_tier, Membership_status	- Membership_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Membership_ID(PK) => Membership_tier, Membership_status	R1) Membership (Membership_ID (PK), Membership_tier, Membership_status)
Refund_Payment	Refund_Payment_ID (PK), Refund_Payment_Method	- Refund_Payment_ID (PK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation”	R1) Refund_Payment (Refund_Payment_ID (PK), Refund_Payment_Method)

			Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Refund_Payment_ID(PK) => Refund_Payment_Method	
Refund_Request	Refund_ID (PK), Refund_Date, Refund_Amount, Refund_Status, Approval_Date, Refund_Notes, Return_ID (FK), Refund_Payment_ID(FK)	- Refund_ID (PK) - Return_ID (FK) - Refund_Payment_ID (FK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Refund_ID(PK) => Refund_Date, Refund_Amount, Refund_Status, Approval_Date, Refund_Note	R1) Refund_Request (Refund_ID (PK), Refund_Date, Refund_Amount, Refund_Status, Approval_Date, Refund_Note)
Return_Request	Return_ID (PK), Return_Date, Return_Reason, Return_Status, Approval_Date, Return_Notes, Customer_account_ID(FK)	- Return_ID (PK) - Customer_account_ID (FK)	Repeating Group (1NF): “ There is no problem about repeating group on this relation ” Partial Dependency (2NF): “There is no problem about Partial Dependency on this relation” Transitive Dependency (3NF): “ There is no problem about Transitive Dependency on this relation ” * Return_ID(PK) => Return_Date, Return_Reason, Return_Status, Approval_Date, Return_Notes	R1) Return_Request (Return_ID (PK), Return_Date, Return_Reason, Return_Status, Approval_Date, Return_Notes)

Relational Schema After Normalization



UML Diagram After Normalization



What we have changed from Project Phase 2

Changed Attributes in UML

1. Remove attributes name “**logs_history**” from Entity name “**Order**”
2. Remove attributes name “**Payment_history**” from Entity name “**Payment**”

Added Entities in UML

1. Add attributes name “**Receipt**” with the attributes (Receipt_ID {PK}, Receipt_DateTime, Quantity, TotalPrice, ChangeDue, TAX_Rate)
2. Add attributes name “**Invoice**” with the attributes (Invoice_ID {PK}, Invoice_Details, Quantity, Original_Price, VAT_Rate, TotalAmount)
3. Add attributes name “**Payment_Method**” with the attributes (Payment_type {PK})
4. Add disjoint (Optional, Or) or (8c type) to “**Payment_method**” with the following Entities and Attributes
 - 4.1 **Payment_Credit_Card** with the Attributes (Credit_Card_Number {PK}, Payment_name, Payment_Description)
 - 4.2 **Payment_QR** with the Attributes (QR_code_ID {PK}, Payment_name, Payment_Description)

Relational Schema

1. Add **Payment_Method** (Payment_type)
2. Add disjoint (8c) from the **Payment_Method**
 - 2.1 **Payment_QR** (Payment_type, QR_code_ID, Payment_name, Payment_description)
 - 2.2 **Payment_Credit_Card** (Payment_type, Credit_Card_number, Payment_name, Payment_description)

Reference

Formal UML Logic (Standard Rules) : <https://www.omg.org/spec/UML/>

Industry Reference: IKEA Thailand – Customer Service and Returns Policy:
<https://www.ikea.com/th/en/customer-service/>

ITCS442_Relational Database Design lect 1-3 Faculty of Information and Communication Technology Mahidol University : Instructor: Asst.Prof. Dr. Charnyote Pluempitiwiriyawej.

AI level 2 from Google Gemini : <https://gemini.google.com>

Operations management of IKEA [SlideShare.com](https://www.slideshare.net): <https://www.slideshare.net>

User registration form <https://family.ikea.co.th/registration>

Employee application:

<https://jobs.smartrecruiters.com/IkanoRetail/744000102921265-business-analyst-finance-business-partner-commercial-finance-ikea-sukhumvit-and-ikea-bangna?trid=c39a8cee-af0e-4db1-9034-acbba85a4a4d>

Delivery service report and Tracking online orders report:

<https://www.ikea.com/mx/en/customer-service/track-manage-order/>

IKEA: A Case Study Through the Lens of Algorithms and Data Structures:

<https://medium.com/@harshada.deshingkar22/ikea-a-case-study-through-the-lens-of-algorithms-and-data-structures-af8de39e47b7>

CaseStudyIkea:

<https://www.scribd.com/document/468047669/Case-Study-Ikea>

System and boundary diagram from draw.io:

https://drive.google.com/file/d/1UqdvIA_Itw7rvLBpeILYLLjwyldW-z_3/view?usp=sharing

UML Diagram and Drafting by Lucid.app:

https://lucid.app/lucidchart/4fb7d03f-d529-43b3-900d-0257bfbe9b52/edit?viewport_loc=-1180%2C-1825%2C10702%2C6397%2CisStbOByeoYb&invitationId=inv_f68534b1-f4bf-4396-adc4-c0e7b2ca4d65